Flipkart Gridlock Hackathon 2.0

Engineering Approach & Predictive Modeling Methodology for Urban Traffic Optimization

Team Name: HyperDrive
Team Members: Pankhuri Srivastava & Swarda Kishor Sawale
Document Status: Final Solution Design Artifact(Approach And Features)

1. Introduction & Problem Formulation

As metropolitan centers scale globally, traffic gridlock remains a primary inhibitor to economic efficiency, micro-logistics timeliness, and consumer travel experiences. Anticipating passenger flow maps before they materialize allows ride-hailing networks and urban routers to balance vehicle supply and actively handle systemic bottlenecks.

In the Flipkart Gridlock Hackathon 2.0, the challenge centers on designing an intelligent system capable of mapping traffic demand arrays across highly dynamic geolocated targets and continuous timelines. The optimization metric tracks a modified variation of the standard coefficient of determination:

$$\text{score} = \max(0, 100 \times R^2\_\text{score}(\text{actual}, \text{predicted}))$$

The final pipeline yields predictions mapped out over a specified matrix of 41,778 rows containing the exact column pairs required: Index and demand, with values bounded tightly inside the normalized interval [0, 1].

2. Unified Pipeline Architecture & Empirical Performance

Our design combines robust predictive power with stable training consistency. Rather than placing full reliance on a singular algorithm, our pipeline constructs an elegant, supervised gradient-boosted ensemble that blends the unique modeling attributes of CatBoostRegressor and LightGBMRegressor.

The end-to-end framework implements a sequential execution structure carefully designed to eliminate data leakage:

- **Chronological Ordering:** To preserve time-series logic, the full dataset is explicitly aligned along a linear chronological axis (time_order) before validation partition blocks are carved out.
- **Leakage-Free Feature Generation:** Imputation tables, spatial clusters, and expanding target indices are built sequentially, drawing statistics only from preceding data boundaries.
- **Bayesian Space Traversal:** An isolated Optuna study backed by an SQLite file optimizes structural hyperparameter configurations over continuous temporal validation splits.
- **Linear Blending & Constraints:** Final inference outputs are generated using a fixed linear combination, passing through a hard clipping layer to prevent out-of-bounds prediction penalties.

Empirical Performance & Validation Scores:

- Baseline LightGBM Regressor R^2: 0.7762
- Baseline CatBoost Regressor R^2: 0.7921
- Final Optimized Ensemble Blend (0.60 × CB + 0.40 × LGBM) R^2: 0.8042

3. Deep Feature Engineering & Extraction Strategy

The core performance of team HyperDrive's solution stems from extensive feature transformations. Raw data fields are extracted and expanded across several operational layers:

3.1. Context-Aware Null Imputation

• Weather & RoadType: Missing entries are systematically resolved using global training modes calculated entirely prior to the validation cuts.
• Temperature: Standard global median mapping introduces forward-looking thermal bias. To avoid this, our pipeline builds a multi-dimensional mapping structure that imputes missing temperatures using the median values computed specifically for that localized hour across the training set.

3.2. Trigonometric Temporal Projections

Linear numerical tracks fail to express daily cycles, treating hour 23 and hour 0 as maximum distances apart. To resolve this, our system maps timestamps onto a continuous circular plane:

$$seconds\_since\_midnight = hour \times 3600 + minute \times 60 + second$$
$$hour\_sin = sin(2\pi \times seconds\_since\_midnight / 86400)$$
$$hour\_cos = cos(2\pi \times seconds\_since\_midnight / 86400)$$

We further augment these projections with contextual markers, including is_rush_hour (covering heavy commute windows: 7-9 AM, 5-8 PM), is_night (10 PM - 5 AM), and an absolute proximity penalty metric (proximity_to_peak) to track continuous operational transitions. Weekly adjustments are managed through cyclic weekday transformations (weekday_sin, weekday_cos) and weekend indicator toggles (is_weekend).

3.3. Unsupervised Geospatial Hotspot Recovery

Raw base-32 string geohashes are mapped to exact float coordinate pairs (lat, lng). Because air-gapped competition execution blocks external package installations, our notebook includes a native pure-Python geohash decoder.

To model urban traffic drawpoints, our pipeline isolates historical training rows matching peak demand thresholds (the top 20% highest values). A localized K-Means clustering loop evaluates cluster variations for k ∈ [3, 6], determining the optimal setup via silhouette score diagnostics. These centers pinpoint heavy congestion epicenters. For each observation, the pipeline calculates:

- **dist_to_epicenter:** The exact Euclidean distance to the nearest verified high-congestion hub.
- **near_epicenter:** A strict binary boundary flag capturing locations inside a tight 0.005 spatial radius.

3.4. High-Dimensional Spatial-Temporal Interactions & Target Encoding

To assist tree cuts in rapidly isolating compounding localized patterns, custom interaction keys are constructed: geohash_4, geohash_5, geohash_hour, and geohash_weekday_hour. Physical roadway constraints are mapped through a specialized traffic-handling index:
road_capacity_index = NumberofLanes × (1 - LargeVehicles)

Standard cross-validated target summaries introduce significant look-ahead leakage in time-series frameworks. To ensure historical consistency, team HyperDrive implements a strict one-step shifted chronological expanding window mean. Demand summaries accumulate progressively over time, ensuring any given row's profile relies purely on past trends.

Unseen geohashes encountered during test-set inference are managed through a robust multi-tiered fallback architecture that downshifts to broader aggregates:
Granular Key (geohash_weekday_hour) → Hour Tile (geohash_hour) → Region Block (geohash_4) → Global Baseline Mean

4. Validation & Operational Core Models

Our optimization pipeline relies on an automated Optuna hyperparameter sweep utilizing Tree-structured Parzen Estimators (TPE) over a sequential 5-fold TimeSeriesSplit. Tuning states are mapped continuously to a persistent SQLite back-end configuration file (optuna_study.db), ensuring fully resumable optimization runs.

Operational Property	CatBoostRegressor Strategy (60%)	LightGBMRegressor Strategy (40%)
Categorical Inputs	Natively handled via internal ordered target statistics.	Encoded using a shared LabelEncoder fit over combined vocabularies.
Loss / Metric	Root Mean Squared Error (RMSE) / R^2 Optimization	Root Mean Squared Error (RMSE) / R^2 Optimization
Overfitting Controls	50-round strict validation early-stopping.	Maximum tree complexity restricted to 63 leaves.
Inference Ensembling	Weighted blend factor: 0.60	Weighted blend factor: 0.40

5. Execution Environment & Artifact Details

The pipeline is written natively inside a single monolithic, production-ready script: Traffic Demand Prediction(Hyper Drive).ipynb. This file handles data ingestion, processes feature engineering layers, tracks hyperparameter progress, and exports the final submission matrix.

- **Core Processing Stack:** Python 3.x, pandas, numpy, scikit-learn.
- **Modeling Engines:** catboost, lightgbm, optuna, sqlite3, shap.